ENTERPRISE DIGITAL BANKING SYSTEM

04

API Documentation

Authentication, conventions, full endpoint reference with request/response examples, error codes and the OpenAPI/Swagger setup.

Document	04 — API Documentation
System	Enterprise Digital Banking System (banking-app) + notification-service
Stack	Java 17 · Spring Boot 4.1 · Spring Security + JWT · Spring Data JPA · MySQL 8 · Apache Kafka 3.8 · Redis 7 · Docker
Repository	Enterprise-Digital-Banking-System (branch `master`)
Version / date	1.0 — 13 October 2026
Author	hemantwint-hey

Status tags used throughout: **BUILT** exists in the code today **PLANNED** part of the target design (roadmap docs 01–09), not yet implemented **GAP** known limitation

Contents

1. 1. Conventions
2. 2. Authentication
3. 3. Customers
4. 4. Accounts
5. 5. Transactions and Ledger
6. 6. Beneficiaries
7. 7. Loans and EMIs
8. 8. Audit and Notifications
9. 9. Errors
10. 10. OpenAPI / Swagger PLANNED
11. 11. Quick-start cURL Script

1. Conventions

Item	Value
Base URL (local)	`http://localhost:8080` · behind Nginx: `https://<host>` PLANNED
Versioning	URI versioning: `/api/v1/...`. Auth endpoints are unversioned: `/auth/...`
Format	JSON request and response bodies, `Content-Type: application/json`
Authentication	`Authorization: Bearer <JWT>` on every endpoint except `/auth/**` (and Swagger UI once added)
Money	Decimal numbers with 2 fraction digits, e.g. `2500.00`
Dates	`LocalDate` → `"2026-10-13"`; `LocalDateTime` → `"2026-10-13T20:15:42.123456"` (server local time)
IDs	Numeric `Long` for all resources except users (`UUID`)
Enums	Sent and returned as upper-case names, e.g. `"SAVINGS"`
Methods	GET read · POST create / action · PUT full update · PATCH state change · DELETE remove
Lists	Return JSON arrays today; paged `Page<T>` with `?page=&size=&sort=` is PLANNED

1.1 Endpoint summary

Method	Path	Purpose	Success
POST	`/auth/register`	Register user	201
POST	`/auth/login`	Log in, get JWT	200
POST	`/api/v1/customers`	Create customer profile	201
GET	`/api/v1/customers`	List customers	200
GET	`/api/v1/customers/{customerId}`	Get customer	200
GET	`/api/v1/customers/user/{userId}`	Get customer by user UUID	200
PUT	`/api/v1/customers/{customerId}`	Update customer	200
PATCH	`/api/v1/customers/{customerId}/kyc`	Complete KYC	200
DELETE	`/api/v1/customers/{customerId}`	Delete customer	204
GET	`/api/v1/customers/search?name=`	Search by name PLANNED	200
POST	`/api/v1/accounts`	Open account	201
GET	`/api/v1/accounts/{accountId}`	Get account	200
GET	`/api/v1/accounts/number/{accountNumber}`	Get by account number	200
GET	`/api/v1/accounts/customer/{customerId}`	Accounts of customer	200
PUT	`/api/v1/accounts/{accountId}`	Update branch / type	200
PATCH	`/api/v1/accounts/{accountId}/freeze`	Freeze account	200
PATCH	`/api/v1/accounts/{accountId}/close`	Close account	200

POST	`/api/v1/transactions/deposit`	Deposit	200
POST	`/api/v1/transactions/withdraw`	Withdraw	200
POST	`/api/v1/transactions/transfer`	Transfer	200
GET	`/api/v1/transactions/{transactionId}`	Get transaction	200
GET	`/api/v1/transactions/reference/{ref}`	Get by reference	200
GET	`/api/v1/transactions/account/{accountId}/history`	Account history	200
GET	`/api/v1/ledger/account/{accountId}`	Ledger of account	200
GET	`/api/v1/ledger/transaction/{transactionId}`	Ledger lines of transaction	200
POST	`/api/v1/beneficiaries`	Add beneficiary	201
GET	`/api/v1/beneficiaries/{beneficiaryId}`	Get beneficiary	200
GET	`/api/v1/beneficiaries/customer/{customerId}`	Beneficiaries of customer	200
PUT	`/api/v1/beneficiaries/{beneficiaryId}`	Rename	200
PATCH	`/api/v1/beneficiaries/{beneficiaryId}/activate`	Activate	200
DELETE	`/api/v1/beneficiaries/{beneficiaryId}`	Remove	204
POST	`/api/v1/loans`	Apply for loan	201
GET	`/api/v1/loans/{loanId}`	Get loan	200
GET	`/api/v1/loans/customer/{customerId}`	Loans of customer	200
PATCH	`/api/v1/loans/{loanId}/approve`	Approve + disburse	200
PATCH	`/api/v1/loans/{loanId}/reject`	Reject	200
GET	`/api/v1/emis/loan/{loanId}`	EMI schedule	200
PATCH	`/api/v1/emis/{emiId}/pay`	Pay EMI	200
GET	`/api/v1/audit`	All audit logs	200
GET	`/api/v1/audit/entity/{entityName}/{entityId}`	Audit of one entity	200
GET	`/api/v1/audit/user/{userId}`	Audit by actor	200
POST	`:8081/api/v1/notifications`	Send notification (notification-service)	201
GET	`:8081/api/v1/notifications/{id}`	Get notification	200
GET	`:8081/api/v1/notifications?userId=`	Notifications of user	200

2. Authentication

2.1 Flow

1. `POST /auth/register` once to create a user (role `CUSTOMER`).
2. `POST /auth/login` with e-mail + password → response contains `token`.
3. Send `Authorization: Bearer <token>` on every `/api/v1/**` call. Tokens expire after `jwt.expiration` (default 24 h); log in again afterwards.
4. Missing / invalid / expired token → `401` with the standard error body.

POST **`/auth/register`** public

Field	Type	Rules
fullName	string	required, 3–100
username	string	required, 3–50
email	string	required, e-mail format, unique
phoneNumber	string	required, `^[6-9]\d{9}$`
password	string	required, ≥ 8
confirmPassword	string	required, must equal password

```
// Request
{ "fullName": "Hemant Mahto", "username": "hemant", "email": "hemant@example.com",
  "phoneNumber": "9876543210", "password": "Secret@123", "confirmPassword": "Secret@123" }
// 201 Created
{ "success": true, "message": "User Registered Successfully" }
```

Errors: `400` validation · `409` “Email already exists” · `401` “Password do not match”

POST **`/auth/login`** public

```
// Request
{ "email": "hemant@example.com", "password": "Secret@123" }
// 200 OK
{ "success": true, "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJoZW1hbnRAZXhhbXBsZS5jb20iLCJpYXQiOjE3...signature" }
```

Errors: `400` validation · `404` “Invalid email id” · `401` “Password didnt match”

2.2 Planned auth endpoints PLANNED

Endpoint	Body (DTO already exists)	Purpose
`POST /auth/refresh`	`RefreshTokenRequest {refreshToken}`	Exchange refresh token for a new short-lived access token

`POST /auth/forgot-password`	`ForgotPasswordRequest {email}`	E-mail a one-time reset token
`POST /auth/reset-password`	`ResetPasswordRequest {token, newPassword, confirmPassword}`	Set a new password with the reset token
`POST /auth/change-password`	`ChangePassowordRequest {currentPassword, newPassword, confirmPassword}`	Authenticated password change
`POST /auth/logout`	—	Revoke refresh token; audit LOGOUT

3. Customers

POST **`/api/v1/customers`** — create the customer profile for an existing user

Field	Rules
firstName, lastName	required, 3–50
dateOfBirth	required, `yyyy-MM-dd`
phoneNumber	required, `^[6-9]\d{9}$` , unique
address	required, ≤ 255
city, state, country	required, ≤ 100
postalCode	required, `^\d{6}$`
email	required — e-mail of the registered user to link

```
// Request
{ "firstName": "Hemant", "lastName": "Mahto", "dateOfBirth": "2000-05-14", "phoneNumber": "9876543210",
  "address": "12 MG Road", "city": "Ranchi", "state": "Jharkhand", "country": "India",
  "postalCode": "834001", "email": "hemant@example.com" }
// 201 Created  (CustomerResponse)
{ "customerId": 1, "firstName": "Hemant", "lastName": "Mahto", "dateOfBirth": "2000-05-14",
  "phoneNumber": "9876543210", "address": "12 MG Road", "city": "Ranchi", "state": "Jharkhand",
  "country": "India", "postalCode": "834001", "kycStatus": "PENDING",
  "createdAt": "2026-10-13T20:10:11.402" }
```

Errors: `404` user not found for e-mail · `409` profile already exists / phone already used

GET **`/api/v1/customers/{customerId}`** · **GET** **`/api/v1/customers/user/{userId}`**

Return `CustomerResponse` (shape above). `404` when not found.

GET **`/api/v1/customers`** — list (`CustomerSummaryResponse[]`)

```
[ { "customerId": 1, "firstName": "Hemant", "lastName": "Mahto", "phoneNumber": "9876543210",
    "city": "Ranchi", "kycStatus": "VERIFIED" } ]
```

PUT **`/api/v1/customers/{customerId}`** — full update

Body: same fields as create except `email` ; names 2–50. `409` if the phone belongs to another customer.

PATCH **/api/v1/customers/{customerId}/kyc** — complete KYC

```
// Request
{ "panNumber": "ABCDE1234F", "aadhaarNumber": "123412341234" }
// 200 OK -> CustomerResponse with "kycStatus": "VERIFIED"
```

Rules: PAN `^[A-Z]{5}[0-9]{4}[A-Z]$`, Aadhaar `^\d{12}$`; `409` if linked to another customer. Side effect: `users.kyc_verified = true`.

DELETE **/api/v1/customers/{customerId}** → `204 No Content`

Resets the user's `kycVerified`. Fails (500, FK violation) if the customer still has accounts/loans GAP.

4. Accounts

POST **/api/v1/accounts**

```
// Request
{ "customerId": 1, "accountType": "SAVINGS", "branch": "Ranchi Main", "openingBalance": 5000.00 }
// 201 Created (AccountResponse)
{ "accountId": 1, "accountNumber": "100483920175", "customerId": 1, "customerName": "Hemant Mahto",
  "accountHolderName": "Hemant Mahto", "branch": "Ranchi Main", "balance": 5000.00,
  "accountType": "SAVINGS", "accountStatus": "ACTIVE",
  "createdAt": "2026-10-13T20:12:00.1", "updatedAt": "2026-10-13T20:12:00.1" }
```

Rules: `customerId`, `accountType` required; `branch` ≤ 100; `openingBalance` ≥ 0. Audit: CREATE.

GET **/api/v1/accounts/{accountId}** · **GET** **/api/v1/accounts/number/{accountNumber}** → AccountResponse (cached in Redis **PLANNED**)

GET **/api/v1/accounts/customer/{customerId}** → `AccountSummaryResponse[]`

```
[ { "accountId": 1, "accountNumber": "100483920175", "accountHolderName": "Hemant Mahto",
    "balance": 5000.00, "accountType": "SAVINGS", "accountStatus": "ACTIVE" } ]
```

PUT **/api/v1/accounts/{accountId}** — `{ "branch": "Ranchi East", "accountType": "CURRENT" }`

PATCH **/api/v1/accounts/{accountId}/freeze** — `{ "reason": "Suspicious activity" }` (required, ≤ 255)

PATCH **/api/v1/accounts/{accountId}/close** — `{ "reason": "Customer request" }`

Errors: `400` "Closed account cannot be modified", "Account is already frozen", "Account balance must be zero before closing"; `404` not found. Audit: UPDATE with old/new status.

5. Transactions and Ledger

POST **/api/v1/transactions/deposit** · **POST** **/api/v1/transactions/withdraw**

```
// Request
{ "accountId": 1, "amount": 2500.00, "remarks": "Salary" }
// 200 OK (TransactionResponse) - deposit
{ "transactionId": 10, "transactionReference": "TXN0004829301746",
  "sourceAccountId": null, "sourceAccountNumber": null,
  "destinationAccountId": 1, "destinationAccountNumber": "100483920175",
  "amount": 2500.00, "transactionType": "DEPOSIT", "transactionStatus": "SUCCESS",
  "balanceAfterTransaction": 7500.00, "remarks": "Salary", "createdAt": "2026-10-13T20:15:42.12" }
```

Rules: `amount` ≥ 1.00, `remarks` ≤ 255; account must be ACTIVE; withdraw requires sufficient balance. Errors: `400`, `404`.

POST **/api/v1/transactions/transfer**

```
// To an account
{ "sourceAccountId": 1, "destinationAccountId": 2, "amount": 1000.00, "remarks": "Rent" }
// Or to a saved beneficiary (exactly one of destinationAccountId / beneficiaryId)
{ "sourceAccountId": 1, "beneficiaryId": 3, "amount": 1000.00 }
// 200 OK
{ "transactionId": 11, "transactionReference": "TXN0918273645012", "sourceAccountId": 1,
  "sourceAccountNumber": "100483920175", "destinationAccountId": 2,
  "destinationAccountNumber": "100920384756", "amount": 1000.00, "transactionType": "TRANSFER",
  "transactionStatus": "SUCCESS", "balanceAfterTransaction": 6500.00, "remarks": "Rent",
  "createdAt": "2026-10-13T20:17:03.88" }
```

Errors (`400`): "Provide exactly one of destinationAccountId or beneficiaryId", "Source and Destination cannot be same", "Insufficent balance", "Account X is FROZEN and cannot be used for transactions", "Beneficiary is PENDING and cannot be used for transfers", "Beneficiary does not belong to the source account holder". `404` for unknown ids. Side effects: 2 ledger entries, Kafka `transaction.completed`, audit. Rate limit 10/min per user **PLANNED**.

GET **/api/v1/transactions/{transactionId}** · **GET** **/api/v1/transactions/reference/{transactionReference}** → TransactionResponse

GET **/api/v1/transactions/account/{accountId}/history** → newest first

```
[ { "transactionId": 11, "transactionReference": "TXN0918273645012", "amount": 1000.00,
    "transactionType": "TRANSFER", "transactionStatus": "SUCCESS", "createdAt": "2026-10-
13T20:17:03.88" } ]
// PLANNED: ?page=0&size=20&sort=createdAt,desc -> { "content": [...], "totalElements": 57,
//          "totalPages": 3, "number": 0, "size": 20 }
```

GET `/api/v1/ledger/account/{accountId}` · GET `/api/v1/ledger/transaction/{transactionId}`


```
[ { "id": 21, "transactionId": 11, "accountId": 1, "accountNumber": "100483920175",
    "entryType": "DEBIT", "amount": 1000.00, "balanceAfterEntry": 6500.00,
    "narration": "Rent", "createdAt": "2026-10-13T20:17:03.9" },
  { "id": 22, "transactionId": 11, "accountId": 2, "accountNumber": "100920384756",
    "entryType": "CREDIT", "amount": 1000.00, "balanceAfterEntry": 3000.00,
    "narration": "Rent", "createdAt": "2026-10-13T20:17:03.9" } ]
```


`/transaction/{id}` returns `404` if the transaction has no ledger lines.

6. Beneficiaries

POST **/api/v1/beneficiaries**

```
// Request
{ "customerId": 1, "beneficiaryAccountId": 2, "nickName": "Mom" }
// 201 Created (BeneficiaryResponse)
{ "id": 3, "nickName": "Mom", "accountId": 2, "status": "PENDING", "createdAt": "2026-10-13T20:20:00" }
```

Errors: `404` customer/account; `400` “This account is already added as a beneficiary”.

GET **/api/v1/beneficiaries/{id}** · **GET** **/api/v1/beneficiaries/customer/{customerId}**

PUT **/api/v1/beneficiaries/{id}** — `{ "nickName": "Mother" }`

PATCH **/api/v1/beneficiaries/{id}/activate** — PENDING → ACTIVE, publishes `beneficiary.activated`; `400` if not PENDING.

DELETE **/api/v1/beneficiaries/{id}** → `204`

7. Loans and EMIs

POST **/api/v1/loans**

```
// Request
{ "customerId": 1, "disbursementAccountId": 1, "loanType": "PERSONAL",
  "principalAmount": 100000.00, "interestRate": 12.00, "tenureMonths": 12 }
// 201 Created (LoanResponse)
{ "id": 1, "customerId": 1, "disbursementAccountNumber": "100483920175", "loanType": "PERSONAL",
  "principalAmount": 100000.00, "interestRate": 12.00, "tenureMonths": 12, "emiAmount": 8884.88,
  "status": "PENDING", "createdAt": "2026-10-13T20:25:00", "approvedAt": null, "closedAt": null }
```

Rules: principal ≥ 1000, interest ≥ 0.01, tenure 1–360. EMI = $P \cdot r \cdot (1+r)^n / ((1+r)^n - 1)$, r = rate/1200.

GET **/api/v1/loans/{loanId}** · **GET** **/api/v1/loans/customer/{customerId}**

PATCH **/api/v1/loans/{loanId}/approve** — credits principal to the disbursement account, status ACTIVE, creates *tenureMonths* EMIs.

PATCH **/api/v1/loans/{loanId}/reject** — status REJECTED.

Both: `400` "Loan is ACTIVE and cannot be approved/rejected" unless PENDING. Intended for MANAGER role **PLANNED**.

GET **/api/v1/emis/loan/{loanId}** — schedule ordered by installment

```
[ { "id": 1, "loanId": 1, "installmentNumber": 1, "dueDate": "2026-11-13", "amount": 8884.88,
    "status": "PENDING", "paidAt": null }, ... ]
```

PATCH **/api/v1/emis/{emiId}/pay** → EmiResponse with `"status": "PAID"`. `400` "EMI is already paid". Paying the last EMI closes the loan.

8. Audit and Notifications

`GET` **`/api/v1/audit`** · **`/api/v1/audit/entity/{entityName}/{entityId}`** · **`/api/v1/audit/user/{userId}`**

```
[ { "id": 55, "performedByUserId": "3f1c...-...", "performedByUsername": "hemant",
    "action": "UPDATE", "entityName": "Account", "entityId": "1", "oldValue": "ACTIVE",
    "newValue": "FROZEN", "description": "Account frozen", "ipAddress": "127.0.0.1",
    "createdAt": "2026-10-13T20:30:00" } ]
```

Restricted to AUDITOR / ADMIN PLANNED.

notification-service (port 8081): `POST` **`/api/v1/notifications`** · `GET` **`/api/v1/notifications/{id}`** · `GET` **`/api/v1/notifications?userId=`**

```
// POST body (NotificationRequest)
{ "userId": "3f1c...", "title": "Transaction Successful", "content": "Your TRANSFER of 1000.00 ...",
  "notificationType": "EMAIL", "notificationPriority": "HIGH", "recipientEmail": "hemant@example.com",
  "idempotencyKey": "b2c6...-eventId" }
// 201 Created, Location: /api/v1/notifications/{notificationId}
```

8.1 Kafka event contracts (asynchronous API)

```
// topic: transaction.completed   key: sourceAccountId
{ "eventId": "uuid", "transactionId": 11, "transactionReference": "TXN0918273645012",
  "transactionType": "TRANSFER", "accountId": 1, "accountNumber": "100483920175", "customerId": 1,
  "userId": "uuid", "email": "hemant@example.com", "amount": 1000.00,
  "balanceAfterTransaction": 6500.00, "occurredAt": "2026-10-13T20:17:03.9" }

// topic: beneficiary.activated   key: customerId
{ "eventId": "uuid", "beneficiaryId": 3, "customerId": 1, "userId": "uuid",
  "email": "hemant@example.com", "nickName": "Mom", "accountNumber": "100920384756",
  "occurredAt": "2026-10-13T20:21:00" }
```

Contract rules: additive changes only (new optional fields); `eventId` is the consumer's idempotency key; renaming a field requires a new topic version.

9. Errors

9.1 Error body

Every error — validation, business rule, authentication, or unexpected — returns the same JSON shape (`ErrorResponse`):

```
{ "timestamp": "2026-10-13T20:40:12.551", "status": 400,
  "error": "amount: Amount must be at least 1.00, accountId: Account id is required",
  "path": "/api/v1/transactions/deposit" }
```

9.2 Status codes

Status	Raised by	Meaning / examples
200 OK	—	Successful read / action
201 Created	—	Register, create customer/account/beneficiary, apply loan
204 No Content	—	Delete customer / beneficiary
400 Bad Request	`MethodArgumentNotValidException` , `BadRequestException`	Validation failure (all field errors joined), business rule violation
401 Unauthorized	`JwtAuthenticationEntryPoint` , `InvalidCredentialsException` , `UnauthorizedException`	Missing/invalid/expired JWT; wrong password; password mismatch on register
403 Forbidden	Spring Security PLANNED	Authenticated but role not allowed
404 Not Found	`ResourceNotFoundException` , `UserNotFoundException`	Unknown id / e-mail
409 Conflict	`UserAlreadyExistsException`	Duplicate e-mail, customer profile, phone, PAN, Aadhaar
413 Payload Too Large	Nginx PLANNED	Body > 1 MB
429 Too Many Requests	Nginx / RateLimitFilter PLANNED	Rate limit hit; `Retry-After` header
500 Internal Server Error	Catch-all handler	Generic message “Something went wrong. Please try again later.” (no stack trace leaked). Also currently returned for optimistic-lock conflicts and malformed JSON GAP

9.3 Error catalogue

Code	Message	Endpoint(s)
400	insufficient balance / Insufficent balance	withdraw, transfer
400	Account <no> is <status> and cannot be used for transactions	deposit, withdraw, transfer
400	Source and Destination cannot be same	transfer

400	Provide exactly one of destinationAccountId or beneficiaryId	transfer
400	Beneficiary is <status> and cannot be used for transfers	transfer
400	Beneficiary does not belong to the source account holder	transfer
400	Closed account cannot be modified	update/freeze/close account
400	Account is already frozen	freeze
400	Account balance must be zero before closing	close
400	This account is already added as a beneficiary	add beneficiary
400	Beneficiary is <status> and cannot be activated	activate
400	Loan is <status> and cannot be approved / rejected	approve, reject
400	EMI is already paid	pay EMI
401	Authentication required: invalid, missing, or expired token	all protected
401	Password didnt match · Password do not match	login · register
404	<Resource> not found with id: <id>	all lookups
404	Invalid email id	login
409	Email already exists	register
409	Customer profile already exists ... / PAN ... / Aadhaar number is already linked ...	customer create/update/KYC

10. OpenAPI / Swagger PLANNED

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.6.0</version>
</dependency>

// SecurityConfig - make docs public
.requestMatchers("/auth/**", "/swagger-ui/**", "/swagger-ui.html", "/v3/api-docs/**").permitAll()

// Optional: JWT "Authorize" button
@OpenAPIDefinition(info = @Info(title = "Enterprise Digital Banking API", version = "v1"))
@SecurityScheme(name = "bearerAuth", type = SecuritySchemeType.HTTP, scheme = "bearer", bearerFormat =
"JWT")
@Configuration public class OpenApiConfig {}

// Per controller
@Tag(name = "Loans", description = "Loan application and approval")
@SecurityRequirement(name = "bearerAuth")
```

URL	Content
`/swagger-ui.html`	Interactive UI with “Try it out”
`/v3/api-docs`	OpenAPI 3 JSON (import into Postman / generate clients)
`/v3/api-docs.yaml`	OpenAPI 3 YAML

11. Quick-start cURL Script

```
BASE=http://localhost:8080
curl -s -X POST $BASE/auth/register -H "Content-Type: application/json" -d '{"fullName":"Hemant Mahto",
  "username":"hemant","email":"hemant@example.com","phoneNumber":"9876543210",
  "password":"Secret@123","confirmPassword":"Secret@123"}'
TOKEN=$(curl -s -X POST $BASE/auth/login -H "Content-Type: application/json" \
  -d '{"email":"hemant@example.com","password":"Secret@123"}' | jq -r .token)
H="Authorization: Bearer $TOKEN"
curl -s -X POST $BASE/api/v1/customers -H "$H" -H "Content-Type: application/json" -d
'{"firstName":"Hemant",
  "lastName":"Mahto","dateOfBirth":"2000-05-14","phoneNumber":"9876543210","address":"12 MG Road",
  "city":"Ranchi","state":"Jharkhand","country":"India","postalCode":"834001","email":"hemant@example.com"}'
curl -s -X POST $BASE/api/v1/accounts -H "$H" -H "Content-Type: application/json" \
  -d '{"customerId":1,"accountType":"SAVINGS","branch":"Main","openingBalance":5000}'
curl -s -X POST $BASE/api/v1/accounts -H "$H" -H "Content-Type: application/json" \
  -d '{"customerId":1,"accountType":"CURRENT","branch":"Main","openingBalance":0}'
curl -s -X POST $BASE/api/v1/transactions/transfer -H "$H" -H "Content-Type: application/json" \
  -d '{"sourceAccountId":1,"destinationAccountId":2,"amount":1000,"remarks":"test"}'
curl -s $BASE/api/v1/ledger/account/1 -H "$H"
```